STUDY OF POST-QUANTUM CRYPTOGRAPHY AND ITS APPLICATIONS IN SPACE-SYSTEMS AREAS

A thesis submitted in partial fulfillment of the requirements for the award of the degree of

M.Tech

in

Computer Science and Engineering

By

VENUGOPAL S (206123032)



COMPUTER SCIENCE AND ENGINEERING
NATIONAL INSTITUTE OF TECHNOLOGY
TIRUCHIRAPPALLI – 620015

MAY 2026

To Mother Nature

To Mathematics - The Queen of all Sciences

BONAFIDE CERTIFICATE

This is to certify that the project titled **STUDY OF POST-QUANTUM CRYPTOGRAPHY AND ITS APPLICATIONS IN SPACE-SYSTEMS AREAS** is a bonafide record of the work done by

VENUGOPAL S (206123032)

in partial fulfillment of the requirements for the award of the degree of **Master of Technology** in **Computer Science and Engineering** of the **NATIONAL INSTITUTE OF TECHNOLOGY, TIRUCHIRAPPALLI**, during the year 2025-26.

Dr. KUNWAR SINGH
Internal Guide

Shri. YOGEEN S HONNAVAR
External Guide

Dr. KUNWAR SINGH
Head of the Department

Project Viva-voce held on _____

Internal Examiner

External Examiner

ABSTRACT

With the advent of the quantum computers, the security of the existing classical cryptographic algorithms (like RSA, ECC etc), which are currently in-use in mission critical systems, are at high risk because these algorithms are based on Integer Factorization or Discrete Log Problem (DLP) which can be easily broken by Shor's algorithm in polynomial time running on a powerful quantum computer. Hence alternate options like Quantum Key Distribution (QKD) and Post-Quantum Cryptography (PQC) are the way forward in future. Among these two options, QKD demands higher costs in terms of new hardware requirements and complexities involved in the migration of legacy systems which have been built over past several decades. On the other hand, for such usecases, PQC is found to be affordable and adaptable with the existing infrastructure without compromising on the security posture. Already threat actors are deploying "Harvest Now, Decrypt Later" (HNDL) Attacks where the legacy encrypted data are collected now and stored for decryption using a powerful quantum computer in future. Hence PQC migration is the need of the hour for all organisations involved in critical work. But PQC transition also depends on the environment and usecase to which it is migrated. Hence it is very essential to conduct a detailed study of the impact of PQC migration on legacy systems without causing any detrimental effects on the existing infrastructure.

In this thesis, our contributions are as follows. In Chapter 1, we give an introduction to the necessary concepts. In Chapter 2, we deal with the preliminaries required for the understanding of the research with the state-of-the-art NIST-Standardized PQC schemes : ML-KEM, ML-DSA and SLH-DSA — analyzing their security, performance trade-offs, and implementation considerations. In Chapter 3, we present the literature survey for our work. In Chapter 4, first, we present our methodology for setting up the implementation system. Second, we execute the NIST standardized PQC algorithms in that setup and record the observations. Third, from the recorded observations, we carry out a detailed performance analysis of PQC algorithms and study their impact during the PQC transition. In Chapter 5, we provide the conclusions and future directions to follow up from this research. Finally, we conclude the thesis with the code attachments and the bibliography.

Keywords : Post-Quantum Cryptography, Harvest Now, Decrypt Later (HNDL), National Institute of Standards and Technology (NIST), Federal Information Processing Standards (FIPS), Learning With Errors (LWE), Module-Lattice-Based Key-Encapsulation Mechanism (ML-KEM), Module-Lattice-Based Digital Signature Algorithm (ML-DSA), Stateless-Hash-based Digital Signature Algorithm (SLH-DSA)

ACKNOWLEDGEMENTS

I would like to thank the following people for their support and guidance without whom the completion of this project in fruition would not be possible.

Dr. KUNWAR SINGH (Internal Guide), Associate Professor and Head, Department of Computer Science and Engineering, NIT Tiruchirappalli, for helping and guiding me throughout the course of this project. Without his guidance and mentorship, I would not have been able to successfully complete this project. His patience and genial attitude is and always will be a source of inspiration to me.

Shri. YOGEEN S HONNAVAR (External Guide), Scientist/Engineer-'SG', Section Head - Information Security, Computers and Information Group (CIG), UR Rao Satellite Centre (URSC), Indian Space Research Organisation (ISRO), Dept. of Space (DoS), Govt. of India, for his valuable support and guidance provided throughout the course of this project.

Dr. E. SIVASANKAR , Dr. B. SHAMEEDHA BEGUM , Dr. A. SANTHANA VIJAYAN, the internal reviewers, for their insight and advice provided during the review sessions.

I am also thankful to all the faculty and staff members of the Department of Computer Science and Engineering at NIT Tiruchirappalli, my parents and my friends for their constant support and help.

Last but not the least, I am also specially thankful to my Seniors and Team Members at CIG/URSC - ISRO for their motivation, guidance and support which greatly helped me to pursue and complete my Masters Degree.

TABLE OF CONTENTS

Title	Page No.
ABSTRACT	i
ACKNOWLEDGEMENTS	ii
TABLE OF CONTENTS	iii
LIST OF TABLES	v
LIST OF FIGURES	vi
CHAPTER 1 INTRODUCTION	1
CHAPTER 2 PRELIMINARIES	4
2.1 NIST Standardized PQC Algorithms	4
2.1.1 FIPS-203 MLKEM	4
2.1.2 FIPS-204 MLDSA	5
2.1.3 FIPS-205 SLH-DSA	6
CHAPTER 3 LITERATURE SURVEY	8
CHAPTER 4 PROPOSED WORK	9
4.1 Experimental Setup	9
4.2 Algorithm Execution	10
4.3 Implementation Results	18
4.3.1 Time Complexity Analysis	18
4.3.2 Space Complexity Analysis	20
4.3.3 Power Usage Analysis	21
4.4 Challenges in Implementing PQC in Real World	22
4.4.1 Operational Risk Identification	22
4.4.2 Readiness Audits	22

4.4.3 Risk Management Frameworks 22
4.4.4 Critical Infrastructure Modernization 23
4.4.5 Governance and Compliance 23
4.4.6 Workforce Capability 23
CHAPTER 5 CONCLUSION AND FUTURE WORK 25
5.1 Summary 25
5.2 Future Work 25
APPENDIX A CODE ATTACHMENTS 27
A.1 Code for MLKEM 27
A.2 Code for MLDSA 30
A.3 Code for SLHDSA 31
A.4 Code for Maven Project POM XML 33
A.5 Code for Visualisation 37
REFERENCES 41

LIST OF TABLES

4.1 Results of ML-KEM 10

4.2 Results of ML-DSA-44 10

4.3 Results of ML-DSA-65 11

4.4 Results of ML-DSA-87 11

4.5 Results of SLHDSA-sha2-128f 12

4.6 Results of SLHDSA-sha2-128s 12

4.7 Results of SLHDSA-sha2-192f 13

4.8 Results of SLHDSA-sha2-192s 13

4.9 Results of SLHDSA-sha2-256f 14

4.10 Results of SLHDSA-sha2-256s 14

4.11 Results of SLHDSA-shake-128f 15

4.12 Results of SLHDSA-shake-128s 15

4.13 Results of SLHDSA-shake-192f 16

4.14 Results of SLHDSA-shake-192s 16

4.15 Results of SLHDSA-shake-256f 17

4.16 Results of SLHDSA-shake-256s 17

4.17 KEM Space Complexity Results 20

4.18 DSS Space Complexity Results 20

LIST OF FIGURES

2.1 Key Encapsulation Mechanism (KEM) Workflow 5
2.2 Digital Signature Scheme (DSS) Workflow 6

4.1 Plot of ML-KEM 18
4.2 Plot of ML-DSA 18
4.3 Plot of SLH-DSA-SHA2 19
4.4 Plot of SLH-DSA-SHAKE 19
4.5 Power Usage Analysis 21

CHAPTER 1

INTRODUCTION

Cryptography helps to protect the data from access by unwanted adversaries. Usually this security assumption is provided by the intractable behaviour of any proven hardness problem in the classical world. But recent developments in quantum algorithms, like the Shor's algorithm, can break these hardness and make them tractable, thus posing a great threat to the present cryptographic methods. Quantum cryptography applies the quantum mechanical principles into the cryptographic algorithms.The security assumption in quantum cryptography is provided by the laws of nature such as the No Cloning Theorem, which are not breakable, and thus provides higher level of security than classical cryptography.

In 1994, Peter Shor [1] [2] showed that prime factorization and discrete logarithm problem can be solved in polynomial time efficiently on a quantum computer. In other words, once a quantum computer becomes a reality classical asymmetric cryptographic algorithms like RSA and ECC will be broken in a few seconds. With the Grover's algorithm introduced by LK Grover in 1996 [3], the strength of classical symmetric cryptographic algorithms are also reduced to half (i.e) quadratic speedup. Hence the existence of all of the classical cryptography hangs in the balance of the viability of a real-world powerful quantum computer. As quantum computing advances rapidly, conventional public-key cryptographic algorithms face the looming threat of being broken by quantum attacks. This has triggered a proactive push toward Quantum Cryptography (QC) and Post-Quantum Cryptography (PQC), focusing on algorithms believed to resist both classical and quantum computational capabilities.

Cryptographic agility is the ability of a system, application, or infrastructure to quickly replace, upgrade, or modify cryptographic algorithms, protocols, keys, or certificates without major disruption to operations. It helps the organisations to adapt to evolving cryptographic requirements in a fast and efficient manner. It is important because cryptographic standards can become outdated or vulnerable over time. Hybrid solutions act as a practical bridge, enabling a gradual and secure transition toward fully post-quantum systems.

"Harvest Now, Decrypt Later" (HNDL) is a cybersecurity threat model where attackers collect and store encrypted data today with the intention of decrypting it in the future when more powerful quantum computers become available. This is especially dangerous for organisations handling with long-term confidential information

like Government and defense sectors.

Currently we are in the Noisy Intermediate-Scale Quantum (NISQ) era where quantum hardware is prone to decoherence and lacks error correction. But ultimately we are working to reach the Fault-Tolerant Quantum Computing (FTQC) era where executing deep quantum circuits required for executing algorithms like Shor's and Grover's for real-world applications is practically possible using a Cryptographically Relevant Quantum Computer (CRQC).

Quantum Key Distribution (QKD) [4] [5], which is under Quantum Cryptography (QC), emerges as a promising alternative for secure key sharing in the quantum era. By exploiting the principles of quantum mechanics [6], such as entanglement, QKD establishes provably secure keys between parties. This is very critical because in future an attacker with a powerful quantum computer could break the currently existing cryptographic algorithms like RSA, Elliptic curve etc. So we have to design new cryptographic algorithms which will be quantum-resistant to these futuristic attacks. The main disadvantage with QKD is that it requires development of totally new setup and hardware. This is a big change to the underlying classical infrastructure which had already been built over the past several decades. Most of the critical federal systems like the ground network systems of space navigation are spread globally and so they are very difficult to be replaced entirely with QKD infrastructure.

Post-Quantum Cryptography (PQC) emerges as an another proactive response. It comprises cryptographic schemes based on mathematical problems believed to be insurmountable even by quantum computers and these algorithms are capable of running on existing classical computers too. Algorithm families such as lattice-based, hash-based, code-based, multivariate, and isogeny-based systems are at the core of ongoing PQC efforts. NIST has already begun updating their standardization practices by initiating the Post-Quantum Cryptography (PQC) Standardization program in 2016 [7]. Among several of the proposed algorithms from various parts of the world , NIST has carefully reviewed all of them and on August-2024 NIST has finalised 3 algorithms among them as the finalized PQC algorithms : ML-KEM as FIPS-203 [8], ML-DSA as FIPS-204 [9] and SLH-DSA as FIPS-205 [10]. The finalists were separated into two groups: lattice-based schemes and hash-based schemes. The lattice-based schemes includes the "CRYSTALS-Kyber" and "CRYSTALS-Dilithium". The hash-based schemes includes the "SPHINCS+".

Despite the promise of PQC, real-world deployment is non-trivial. Critical challenges include increased computational and bandwidth costs — especially for IoT and embedded systems, complex integration into existing protocols like TLS, and emerging implementation vulnerabilities such as side-channel attacks. Hence the PQC transition has to be studied carefully depending upon the environment to

which its going to be migrated and all the procedures have to be carried out after careful testing and in a phased manner to prevent any service disruptions and align with the Service Level Agreements (SLA's) of the organisation.

In this thesis, our contributions are as follows. In Chapter 2, we deal with the preliminaries required for the understanding of the research with the state-of-the-art NIST-Standardized PQC schemes : ML-KEM, ML-DSA and SLH-DSA — analyzing their security, performance trade-offs, and implementation considerations. In Chapter 3, we present the literature survey for our work. In Chapter 4, first, we present our methodology for setting up the implementation system. Second, we execute the NIST standardized PQC algorithms in that setup and record the observations. Third, from the recorded observations, we carry out a detailed performance analysis of PQC algorithms and study their impact during the PQC transition. In Chapter 5, we provide the conclusions and future directions to follow up from this research. Finally, we conclude the thesis with the code attachments and the bibliography.

CHAPTER 2

PRELIMINARIES

2.1 NIST Standardized PQC Algorithms

The NIST PQC initiative [7] develops new cryptographic algorithms designed to secure digital information against future attacks by quantum computers. Because large-scale quantum computers can break widely used public-key systems, NIST is standardizing quantum-resistant algorithms for global deployment.

On 12th August 2024, NIST has released its first finalized set of three PQC standards. These new cryptographic algorithms, which serve as Federal Information Processing Standards (FIPS), include the following key details :

2.1.1 FIPS-203 MLKEM

ML-KEM (known as CRYSTALS-Kyber) [8] is a lattice-based key encapsulation mechanism standardized by NIST as FIPS 203. It provides a quantum-resistant way to establish shared secret keys between parties.

The algorithm is based on the hardness of the Module Learning With Errors (MLWE) problem [11], making it resistant to attacks from both classical and quantum computers.

ML-KEM is designed to replace current algorithms like RSA and Elliptic Curve Diffie-Hellman (ECDH) in TLS and other protocols, ensuring that encrypted communications remain secure even in the quantum era. Fig. 2.1 shows the detailed workflow of KEM.

Security Levels

1. ML-KEM-512 (NIST Category 1, 128-bit security level)
2. ML-KEM-768 (NIST Category 3, 192-bit security level)
3. ML-KEM-1024 (NIST Category 5, 256-bit security level)

How ML-KEM Works

1. Key Generation : Alice generates a key pair consisting of a public key and a secret key, based on randomly sampled polynomials in a special ring. Output: Public Key, Secret Key

2. Encapsulation : Bob takes Alice's public key, generates a random value, and uses it to derive a shared secret. He also creates a ciphertext to send to Alice. Input: Alice's Public Key Output: Ciphertext, Shared Secret

3. Decapsulation : Alice uses her secret key and the ciphertext from Bob to recover the same shared secret, without any further communication. Input: Alice's Secret Key, Bob's Cipher Text Output: Shared Secret

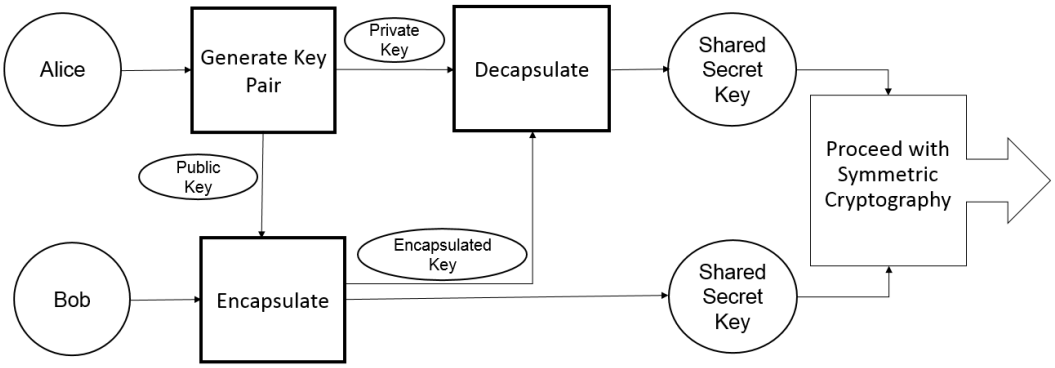


Figure 2.1: Key Encapsulation Mechanism (KEM) Workflow

2.1.2 FIPS-204 MLDSA

ML-DSA (known as CRYSTALS-Dilithium) [9] is a lattice-based digital signature algorithm standardized by NIST as FIPS 204. It provides a quantum-resistant way to generate and verify digital signatures.

Like ML-KEM, the algorithm is based on the hardness of the Module Learning With Errors (MLWE) problem, making it resistant to attacks from both classical and quantum computers.

ML-DSA is designed to replace current digital signature algorithms like RSA and ECDSA (Elliptic Curve Digital Signature Algorithm) in applications requiring document signing, code signing, certificate validation, and more. Fig. 2.2 shows the detailed workflow of DSS.

Security Levels

1. ML-DSA-44 (NIST Category 2, 128-bit security level)
2. ML-DSA-65 (NIST Category 3, 192-bit security level)
3. ML-DSA-87 (NIST Category 5, 256-bit security level)

How ML-DSA Works

1. Key Generation : The signer generates a key pair consisting of a public key that will be published and a secret key that is kept confidential. Output: Public Key, Secret Key

2. Signing : The signer uses their secret key to create a digital signature for a message, which serves as proof of authenticity. Input: Message, Secret Key Output: Digital Signature

3. Verification : Anyone can verify the signature's authenticity using the signer's public key, confirming the message hasn't been altered. Input: Message, Signature, Public Key Output: Valid or Invalid



Figure 2.2: Digital Signature Scheme (DSS) Workflow

2.1.3 FIPS-205 SLH-DSA

SLH-DSA (known as SPHINCS+) [10] is a hash-based digital signature algorithm standardized by NIST as FIPS 205. Unlike lattice-based algorithms, SLH-DSA's security is based on the properties of cryptographic hash functions, which are believed to be resistant to both classical and quantum attacks.

SLH-DSA offers multiple parameter sets with different tradeoffs between speed, signature size, and security level. The "f" variants prioritize speed, while the "s" variants produce smaller signatures at the cost of performance.

SLH-DSA operations are significantly slower than ML-KEM and ML-DSA, especially at higher security levels. This is a characteristic of hash-based signature schemes. In production environments, this would be optimized with native code implementations.

Security Levels

1. SLH-DSA-128 (128-bit security level)
2. SLH-DSA-192 (192-bit security level)
3. SLH-DSA-256 (256-bit security level)

How SLH-DSA Works

1. Key Generation : The signer generates a key pair using a combination of cryptographic hash functions. This process creates a public key that will be published and a secret key kept confidential. Output: Public Key, Secret Key

2. Signing : Using their secret key, the signer creates a digital signature for a message through a complex combination of WOTS+ (Winternitz One-Time Signature Plus) one-time signatures, FORS (Forest of Random Subsets) trees, and hypertree structures. Input: Message, Secret Key Output: Digital Signature

3. Verification : Anyone can verify the signature's authenticity using the signer's public key and hash functions, confirming the message hasn't been altered and was signed by the holder of the secret key. Input: Message, Signature, Public Key Output: Valid or Invalid

Other Sub Variants of SLHDSA

1. f - (Fast) Optimized for speed
2. s - (Small) Optimized for signature size
3. SHA-2 - Standard NIST hash function
4. SHAKE - SHA-3 derived function

CHAPTER 3

LITERATURE SURVEY

[12] reviews major PQC algorithms such as lattice-based, code-based, hash-based, and isogeny-based cryptography, while analyzing the NIST standardization process and challenges posed by quantum computing to traditional encryption systems like RSA and ECC. It further discusses migration strategies, emphasizing hybrid cryptographic approaches and phased transition methods to ensure secure and compatible adoption of quantum-resistant systems.

[13] examines the implications of PQC on existing communication networks, highlighting NIST-standardized algorithms while emphasizing the need for secure migration from classical cryptography.

[14] investigates the implementation of NIST-selected PQC algorithms for satellite communication networks, focusing on secure boot and over-the-air firmware update systems on RISC-V platforms. It further analyzes the performance impact and feasibility of deploying PQC in satellite systems to enhance resilience against emerging quantum and cybersecurity threats.

[15] analyzes the impact of quantum computing threats on Public Key Infrastructure (PKI) systems used in production environments and proposes security recommendations for transitioning toward PQC solutions. It emphasizes crypto-agility, hybrid cryptographic schemes, certificate management improvements, and the adoption of quantum-resistant algorithms to ensure secure and scalable PKI deployment in the post-quantum era.

[16] examines the vulnerability of U.S. financial payment infrastructures to HNDL attacks caused by future quantum computers, and proposes the adoption PQC standards for secure financial transactions. It further recommends a phased hybrid migration strategy using NIST-standardized algorithms to ensure scalability, compliance, and long-term resilience of financial systems against quantum threats.

CHAPTER 4

PROPOSED WORK

4.1 Experimental Setup

The experimental setup for current PQC evaluation is designed for the capture and analysis of various traces of parameters from the performed cryptographic operations. During the cryptographic operation, the traces from the host target device is recorded in the console logs.

For our experimental setup, we have used a Laptop with hardware specifications of Intel(R) Core(TM) i7-4500U CPU @ 1.8/2.4 GHz (x_64 based), 8 GB of physical RAM and 64-bit Operating System.

In software stack layer, we have used Java-SDK-21 for the cryptographic collections and Python 3.1.3 for data visualisation. Further, required dependencies are handled with the package managers ”Maven” and ”Pip” for Java and Python respectively. All the cryptographic modules are executed with only a single Java Main thread (i.e) Multi-threading is avoided in-order to have an uniform runtime environment.

In order to get clear observations with high precisions, execution of the algorithms over small input texts is avoided since these runs will result in near similar values and so these values cannot be used for further performance studies. Hence we run the specific algorithms against varied and different files, ranging from 50 MB to 900 MB in size. Due to limitations in the present hardware setup, we have limited the maximum cap till 900 MB, which is sufficient and provides with enough samples for our testing and analysis.

The workflow process is same as that of depicted in Fig. 2.1 and Fig. 2.2.

4.2 Algorithm Execution

Table. 4.1 shows the observed execution times for ML-KEM-512, ML-KEM-768, and ML-KEM-1024.

Table 4.1: Results of ML-KEM

File Size (in MB)	Overall Time taken for ML-KEM-512 (in seconds)	Overall Time taken for ML-KEM-768 (in seconds)	Overall Time taken for ML-KEM-1024 (in seconds)
50	1.134	1.277	1.129
100	1.257	1.519	1.312
200	1.565	1.594	1.857
300	1.956	2.027	2.118
400	2.348	2.538	2.705
500	2.854	3.098	3.112
600	3.125	3.248	3.817
700	3.768	3.513	4.034
800	4.12	4.103	4.705
900	4.336	4.87	4.981

Table. 4.2, Table. 4.3 and Table. 4.4 show the observed execution times for ML-DSA-44, ML-DSA-65 and ML-DSA-87 respectively.

Table 4.2: Results of ML-DSA-44

File Size (in MB)	Time taken for Key Generation (in seconds)	Time taken for Signing (in seconds)	Time taken for Verifying (in seconds)	Overall Time (in seconds)
50	0.059	0.428	0.288	0.887
100	0.038	0.682	0.589	2.086
200	0.062	1.385	1.166	3.041
300	0.057	2.041	1.769	4.45
400	0.041	2.572	2.461	5.749
500	0.055	3.984	3.096	7.139
600	0.042	3.824	3.557	7.428
700	0.039	4.477	4.03	8.55
800	0.037	5.273	4.287	9.601
900	0.04	6.229	4.937	11.21

Table 4.3: Results of ML-DSA-65

File Size (in MB)	Time taken for Key Generation (in seconds)	Time taken for Signing (in seconds)	Time taken for Verifying (in seconds)	Overall Time (in seconds)
50	0.086	0.581	0.295	1.084
100	0.041	0.728	0.572	3.205
200	0.054	1.359	1.189	2.952
300	0.041	2.421	1.79	4.797
400	0.04	2.762	2.45	5.988
500	0.048	3.411	2.99	6.452
600	0.044	3.743	3.274	7.065
700	0.051	4.606	3.99	8.651
800	0.063	5.061	4.58	9.71
900	0.049	6.123	5.403	11.57

Table 4.4: Results of ML-DSA-87

File Size (in MB)	Time taken for Key Generation (in seconds)	Time taken for Signing (in seconds)	Time taken for Verifying (in seconds)	Overall Time (in seconds)
50	0.065	0.457	0.349	0.969
100	0.066	0.771	0.566	1.609
200	0.067	1.363	1.131	2.901
300	0.048	2.062	1.798	4.492
400	0.047	2.841	2.417	6.064
500	0.06	3.448	3.0	6.512
600	0.045	3.863	3.454	7.366
700	0.047	4.501	4.356	8.908
800	0.049	5.352	4.754	10.16
900	0.084	6.411	5.265	11.763

Table. 4.5 and Table. 4.6 show the observed execution times for SLHDSA-sha2-128f and SLHDSA-sha2-128s respectively.

Table 4.5: Results of SLHDSA-sha2-128f

File Size (in MB)	Time taken for Key Generation (in seconds)	Time taken for Signing (in seconds)	Time taken for Verifying (in seconds)	Overall Time (in seconds)
50	0.156	1.278	0.39	1.84
100	0.172	1.782	0.765	2.734
200	0.135	3.047	1.532	4.714
300	0.11	4.359	2.203	6.672
400	0.109	5.672	2.953	8.75
500	0.094	6.922	4.0	11.016
600	0.094	7.599	4.343	12.067
700	0.109	8.849	5.388	14.346
800	0.109	10.237	5.453	15.83
900	0.094	11.691	6.657	18.458

Table 4.6: Results of SLHDSA-sha2-128s

File Size (in MB)	Time taken for Key Generation (in seconds)	Time taken for Signing (in seconds)	Time taken for Verifying (in seconds)	Overall Time (in seconds)
50	0.328	3.437	0.391	4.156
100	0.437	4.316	0.782	5.535
200	0.391	5.371	1.547	7.324
300	0.359	6.565	2.203	9.143
400	0.328	7.877	2.969	11.19
500	0.344	9.209	3.985	13.538
600	0.344	9.857	4.328	14.529
700	0.328	11.241	4.672	16.241
800	0.406	12.754	5.5	18.66
900	0.406	14.202	5.734	20.342

Table. 4.7 and Table. 4.8 show the observed execution times for SLHDSA-sha2-192f and SLHDSA-sha2-192s respectively.

Table 4.7: Results of SLHDSA-sha2-192f

File Size (in MB)	Time taken for Key Generation (in seconds)	Time taken for Signing (in seconds)	Time taken for Verifying (in seconds)	Overall Time (in seconds)
50	0.07	0.957	0.385	1.419
100	0.091	1.818	0.771	2.689
200	0.075	3.003	1.661	4.745
300	0.062	4.747	2.306	7.123
400	0.06	5.612	3.039	8.719
500	0.063	6.943	4.035	11.047
600	0.062	7.629	4.455	12.152
700	0.058	8.949	5.285	14.301
800	0.078	10.263	5.592	15.939
900	0.068	11.887	6.411	18.374

Table 4.8: Results of SLHDSA-sha2-192s

File Size (in MB)	Time taken for Key Generation (in seconds)	Time taken for Signing (in seconds)	Time taken for Verifying (in seconds)	Overall Time (in seconds)
50	0.64	3.594	0.391	4.625
100	0.453	4.329	0.75	5.532
200	0.453	5.609	1.485	7.563
300	0.5	6.969	2.25	9.735
400	0.437	8.368	3.14	11.945
500	0.437	9.625	3.943	14.005
600	0.421	10.244	4.359	15.04
700	0.454	11.473	4.625	16.552
800	0.438	13.043	6.006	19.487
900	0.453	14.522	5.734	20.709

Table. 4.9 and Table. 4.10 show the observed execution times for SLHDSA-sha2-256f and SLHDSA-sha2-256s respectively.

Table 4.9: Results of SLHDSA-sha2-256f

File Size (in MB)	Time taken for Key Generation (in seconds)	Time taken for Signing (in seconds)	Time taken for Verifying (in seconds)	Overall Time (in seconds)
50	0.083	1.318	0.571	1.982
100	0.105	2.228	1.141	3.484
200	0.052	4.026	2.253	6.338
300	0.06	5.973	3.367	9.407
400	0.058	7.776	4.455	12.295
500	0.049	9.847	5.524	15.426
600	0.055	11.029	5.961	17.05
700	0.056	12.767	7.437	20.266
800	0.064	14.663	7.917	22.651
900	0.067	16.612	9.048	25.733

Table 4.10: Results of SLHDSA-sha2-256s

File Size (in MB)	Time taken for Key Generation (in seconds)	Time taken for Signing (in seconds)	Time taken for Verifying (in seconds)	Overall Time (in seconds)
50	0.315	2.848	0.505	3.674
100	0.385	3.653	1.167	5.212
200	0.33	5.652	2.118	8.106
300	0.342	7.591	3.009	10.947
400	0.395	9.677	3.961	14.037
500	0.391	11.514	4.93	16.84
600	0.328	12.699	5.549	18.582
700	0.339	14.653	6.378	21.374
800	0.325	16.638	7.881	24.849
900	1.031	19.495	9.162	29.698

Table. 4.11 and Table. 4.12 show the observed execution times for SLHDSA-shake-128f and SLHDSA-shake-128s respectively.

Table 4.11: Results of SLHDSA-shake-128f

File Size (in MB)	Time taken for Key Generation (in seconds)	Time taken for Signing (in seconds)	Time taken for Verifying (in seconds)	Overall Time (in seconds)
50	0.132	0.785	0.348	1.274
100	0.062	1.369	0.712	2.15
200	0.06	2.539	1.397	4.001
300	0.057	3.684	2.47	6.216
400	0.06	4.828	2.862	7.756
500	0.064	5.962	3.588	9.62
600	0.077	6.603	3.943	10.634
700	0.06	7.555	4.522	12.143
800	0.073	9.004	4.89	13.973
900	0.062	10.0	5.714	15.784

Table 4.12: Results of SLHDSA-shake-128s

File Size (in MB)	Time taken for Key Generation (in seconds)	Time taken for Signing (in seconds)	Time taken for Verifying (in seconds)	Overall Time (in seconds)
50	0.37	2.603	0.351	3.327
100	0.349	3.201	0.627	4.181
200	0.353	4.305	1.15	5.812
300	0.401	5.43	2.333	8.168
400	0.356	6.564	2.616	9.54
500	0.4	7.742	3.544	11.691
600	0.448	8.465	3.613	12.532
700	0.354	9.337	4.886	14.581
800	0.351	10.653	4.836	15.846
900	0.351	11.846	5.595	17.797

Table. 4.13 and Table. 4.14 show the observed execution times for SLHDSA-shake-192f and SLHDSA-shake-192s respectively.

Table 4.13: Results of SLHDSA-shake-192f

File Size (in MB)	Time taken for Key Generation (in seconds)	Time taken for Signing (in seconds)	Time taken for Verifying (in seconds)	Overall Time (in seconds)
50	0.064	0.806	0.324	1.199
100	0.065	1.34	0.734	2.145
200	0.062	2.475	1.35	3.893
300	0.064	3.603	1.96	5.633
400	0.065	4.742	2.796	7.608
500	0.069	5.943	3.5	9.518
600	0.067	6.651	4.248	10.972
700	0.098	7.699	4.607	12.414
800	0.069	8.792	5.203	14.069
900	0.063	10.167	5.964	16.2

Table 4.14: Results of SLHDSA-shake-192s

File Size (in MB)	Time taken for Key Generation (in seconds)	Time taken for Signing (in seconds)	Time taken for Verifying (in seconds)	Overall Time (in seconds)
50	0.627	3.807	0.338	4.782
100	0.363	4.04	0.643	5.05
200	0.363	5.234	1.273	6.874
300	0.36	6.32	1.761	8.446
400	0.406	7.344	2.265	10.015
500	0.359	8.422	2.86	11.641
600	0.36	9.266	3.193	12.819
700	0.343	10.235	4.002	14.58
800	0.359	11.635	5.027	17.021
900	0.453	13.176	5.311	18.941

Table. 4.15 and Table. 4.16 show the observed execution times for SLHDSA-shake-256f and SLHDSA-shake-256s respectively.

Table 4.15: Results of SLHDSA-shake-256f

File Size (in MB)	Time taken for Key Generation (in seconds)	Time taken for Signing (in seconds)	Time taken for Verifying (in seconds)	Overall Time (in seconds)
50	0.088	1.411	0.325	1.831
100	0.112	1.521	0.639	2.277
200	0.081	2.677	1.302	4.066
300	0.09	3.726	1.938	5.759
400	0.08	4.894	2.569	7.548
500	0.08	6.134	3.508	9.727
600	0.081	6.797	4.202	11.085
700	0.082	7.859	4.835	12.781
800	0.075	9.023	4.865	13.97
900	0.087	10.38	5.904	16.379

Table 4.16: Results of SLHDSA-shake-256s

File Size (in MB)	Time taken for Key Generation (in seconds)	Time taken for Signing (in seconds)	Time taken for Verifying (in seconds)	Overall Time (in seconds)
50	0.639	3.963	0.298	4.904
100	0.451	4.483	0.663	5.601
200	0.476	5.728	1.286	7.495
300	0.454	6.772	2.332	9.563
400	0.473	7.88	2.314	10.672
500	0.46	9.211	2.971	12.648
600	0.571	9.812	3.248	13.638
700	0.458	11.198	4.195	15.856
800	0.465	12.137	5.175	17.784
900	0.46	13.324	6.05	19.838

4.3 Implementation Results

4.3.1 Time Complexity Analysis

The implementation of the PQC algorithms are carried out in the experimental setup and their corresponding observations are noted. Fig. 4.1, Fig. 4.2, Fig. 4.3 and Fig. 4.4 represent the plots of the overall time taken by the respective NIST PQC algorithms.

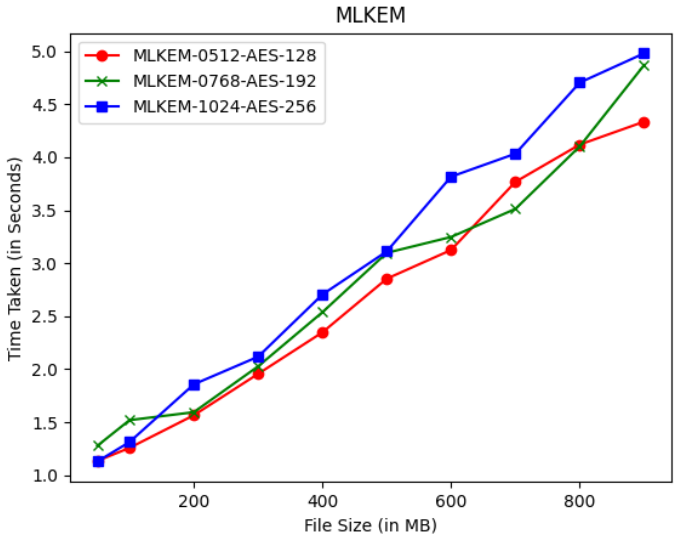


Figure 4.1: Plot of ML-KEM

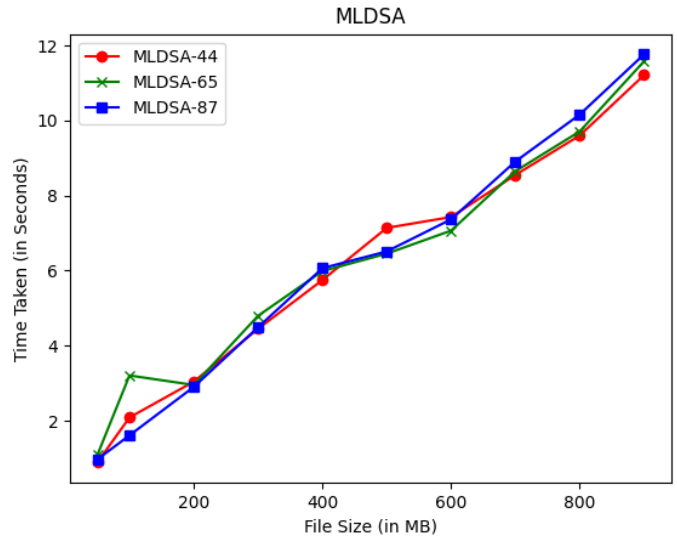


Figure 4.2: Plot of ML-DSA

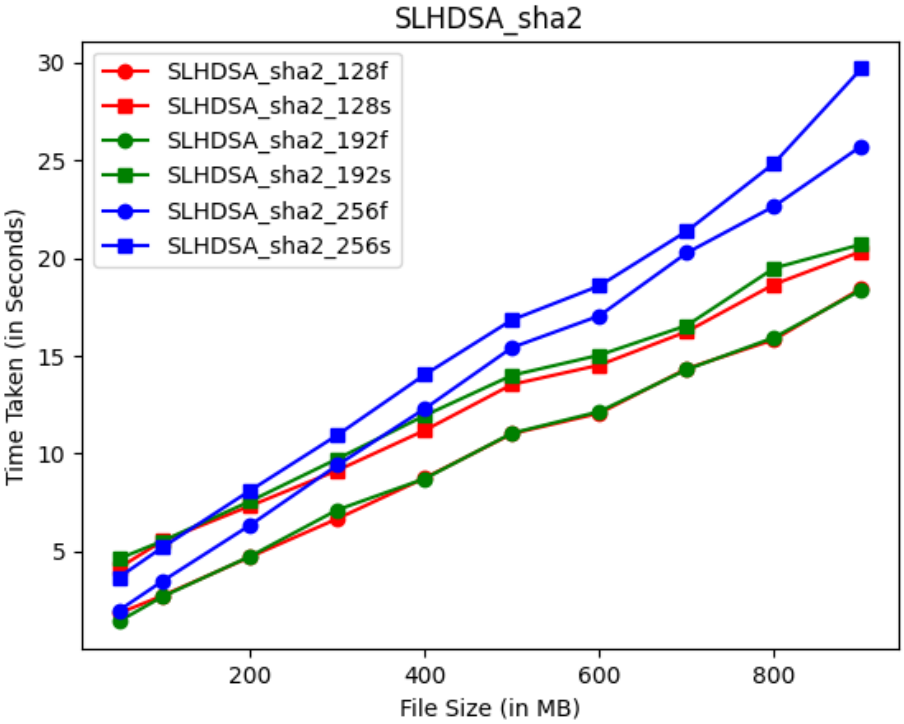


Figure 4.3: Plot of SLH-DSA-SHA2

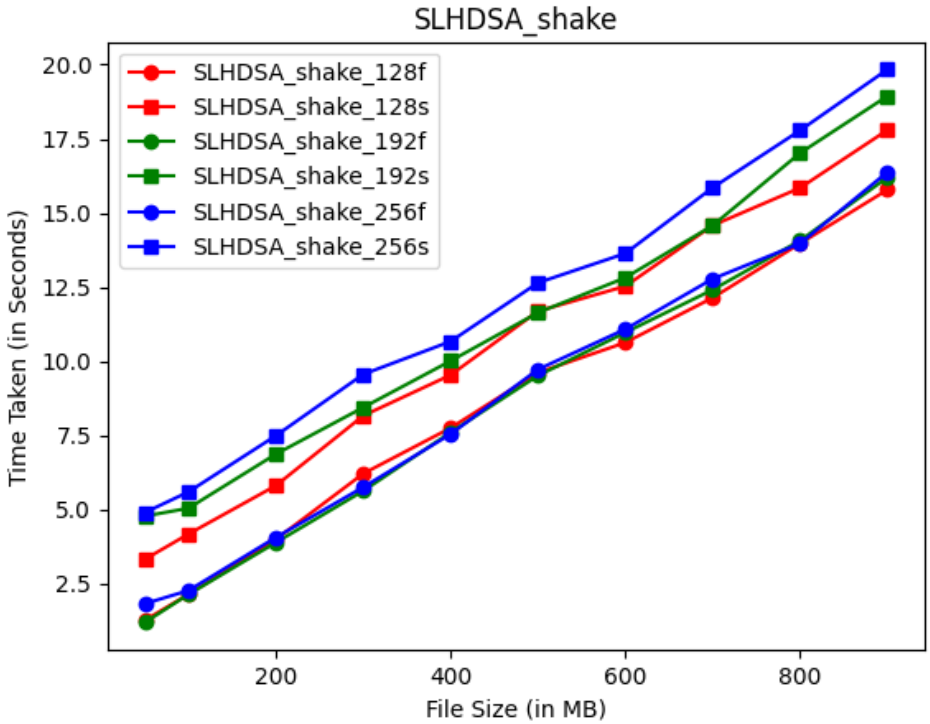


Figure 4.4: Plot of SLH-DSA-SHAKE

4.3.2 Space Complexity Analysis

Table. 4.17 and Table. 4.18 show the space needed during the required stage of the cryptographic operation in KEM and DSS schemes respectively. In Table. 4.18, the SLH-DSA sizes are the same for both SHA2 and SHAKE.

Table 4.17: KEM Space Complexity Results

KEM Algorithm	Public Key length	Private Key length	Length of encapsulated shared secret key
ML-KEM-512	822 B	1730 B	768 B
ML-KEM-768	1206 B	2498 B	1088 B
ML-KEM-1024	1590 B	3266 B	1568 B

Table 4.18: DSS Space Complexity Results

DSS Algorithm	Signature Size (in Bytes)
ML-DSA-44	2420
ML-DSA-65	3309
ML-DSA-87	4627
SLH-DSA-128f	17088
SLH-DSA-128s	7856
SLH-DSA-192f	35664
SLH-DSA-192s	16224
SLH-DSA-256f	49856
SLH-DSA-256s	29792

4.3.3 Power Usage Analysis

Fig. 4.5 represents the power consumed during execution of the PQC algorithm. On average during nominal usage, the discharge rate from the battery stands at 10W. During execution, this value reaches to an average of 20W. Thus we can calculate **Total Energy consumed = Average Power x Overall Time Taken**.

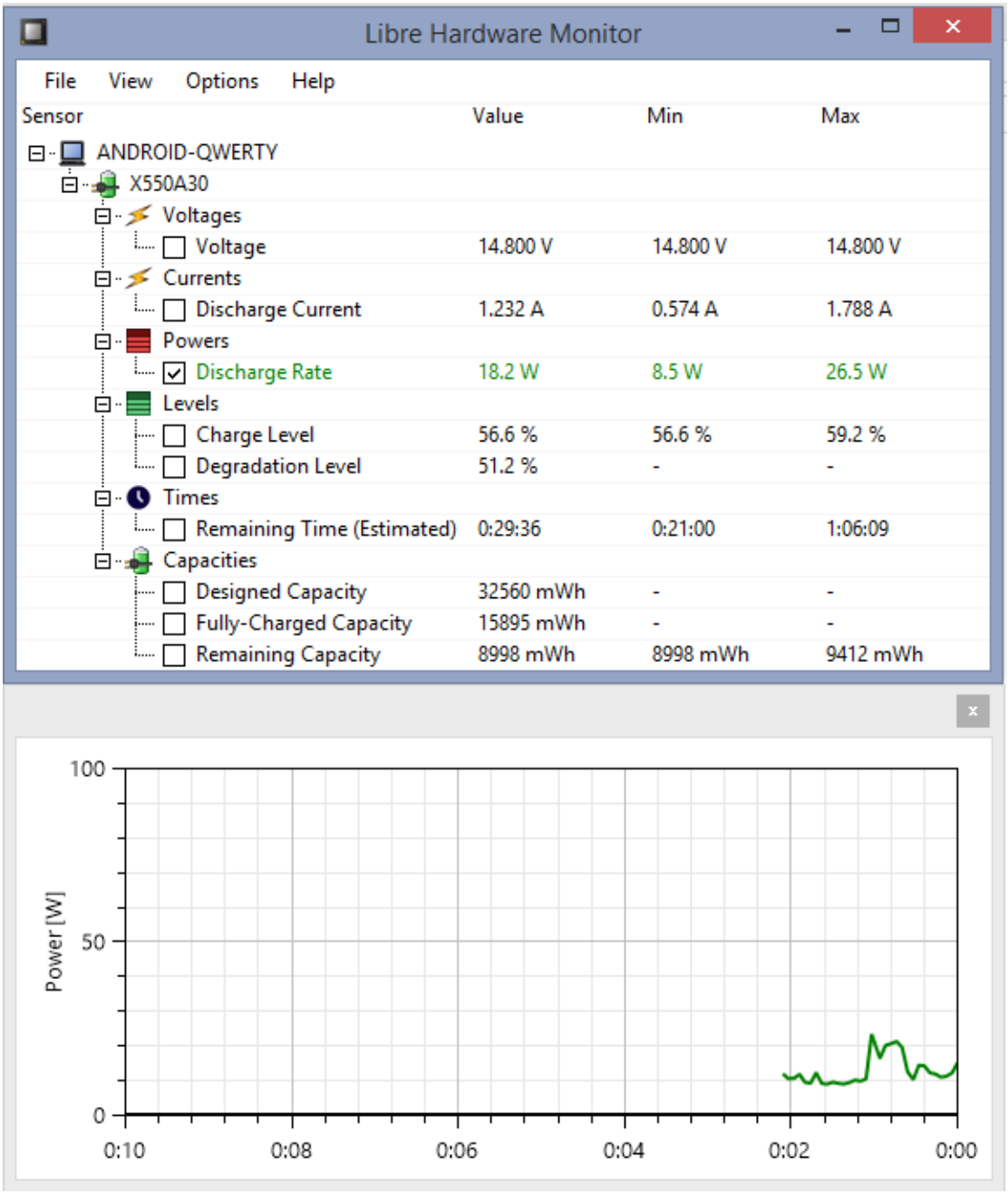


Figure 4.5: Power Usage Analysis

4.4 Challenges in Implementing PQC in Real World

Transitioning to a quantum-secure environment requires an institution-wide strategy that integrates security into daily operations. This road map begins with operational risk identification and readiness audits to inventory vulnerabilities. These findings are embedded into risk management frameworks, guiding critical infrastructure modernization. Crucially, organizations must align governance and compliance with evolving standards, build necessary workforce capability, and implement quantum-secured inter-connectivity to ensure long-term resilience against future decryption threats.

4.4.1 Operational Risk Identification

A robust quantum security strategy begins by assessing operational risk. Institutions must identify emerging attack surfaces by mapping all systems relying on quantum-vulnerable algorithms for migration. Data with long-term confidentiality requirements is immediately vulnerable to the HNDL threat model. Critical data are at high risk and so requires immediate PQC or QKD implementation. Confidential data are at moderate risk and so subject to lingering HNDL exposure. Cryptographic vulnerabilities in vendor systems and external channels compound institutional risk and must be integrated into comprehensive audits. Risk assessment must be continuous, evolving alongside hardware capabilities and threat timelines.

4.4.2 Readiness Audits

While risk identification assesses the threat landscape, readiness audits evaluate infrastructure maturity. They transition the strategy from abstract risk prioritization to concrete technical assessment. The audit begins by cataloging cryptographic dependencies across all systems and workflows—inventorying every algorithm, library, and communication channel currently in use. Assets are then subjected to a gap analysis, measuring the distance between current infrastructure and target quantum-safe standards, such as NIST PQC guidelines. This technical baseline converts high-level risks into a migration road map. A rigorous readiness audit ensures that subsequent governance and remediation decisions are driven by factual system inventories rather than speculation.

4.4.3 Risk Management Frameworks

Quantum risk must not be managed in isolation. It requires integration into established Enterprise Risk Management (ERM) structures to ensure institutional accountability. The outputs from risk identification and readiness audits must be em-

bedded within ERM, assigning clear ownership, reporting lines, and escalation paths for all identified vulnerabilities. Institutional resilience should be stress-tested against defined quantum-threat scenarios, modeling both accelerated hardware timelines and worst-case cryptographic compromise. The objective is not to eliminate quantum risk entirely, but to govern it in a structured, measurable, and risk-proportionate manner that aligns with overall institutional strategy.

4.4.4 Critical Infrastructure Modernization

Translating readiness audits and risk frameworks into action requires the systematic migration of cryptographic infrastructure toward quantum-secure standards. Institutions must execute a prioritized, time-bound roadmap replacing vulnerable algorithms with NIST-approved PQC schemes, specifically ML-KEM, ML-DSA, and SLH-DSA. To maintain operational stability, deploy a hybrid transition—running classical and post-quantum algorithms in parallel. This ensures interoperability while mitigating service disruption. Finally, prioritize cryptographic agility. Future systems must be architected to allow the swapping of cryptographic primitives without full infrastructure rebuilds, ensuring long-term adaptability as standards evolve. Migration is a multi-year strategic imperative; institutional resilience depends on the discipline of this transition.

4.4.5 Governance and Compliance

Regulatory landscapes for quantum security are evolving rapidly. Institutions must align internal policies with global standards like NIST proactively, rather than waiting for compliance to become mandatory. Quantum security demands fiduciary oversight. This requires visible governance, with clear accountability assigned to risk committees and regular reporting to senior leadership. Institutional policies must be codified across the full supply chain: updating vendor procurement standards, internal cryptographic policies, and third-party risk assessments to mandate quantum-safe readiness. Governance is the vital mechanism that ensures quantum security investments are sustained, prioritized, and enforceable across the entire institution.

4.4.6 Workforce Capability

Technology alone is not sufficient. Institutions must build the internal expertise needed to understand, implement, and sustain quantum-safe security over time. Risk, compliance, and technology teams must understand the quantum threat landscape and its direct relevance to their roles and responsibilities. Deeper technical capability in post-quantum cryptography, quantum communication, and secure system design must be cultivated through targeted programs and external partnerships. IT, risk, legal, and

business functions must be aligned around a shared quantum security roadmap, with clear roles and accountability at each stage of the transition. The quantum transition is as much an organizational challenge as a technical one. Workforce capability determines whether institutions can execute their migration strategies effectively within the relevant timeline.

CHAPTER 5

CONCLUSION AND FUTURE WORK

5.1 Summary

Through this work, we have performed a structured analysis of the various NIST standardized PQC algorithms on the pre-setup environment.

Thus this thesis explored the viability of transitioning towards the NIST PQC algorithms, aiming to increase accessibility in this field. The results of this research is focussed on evaluating the effectiveness of PQC execution on ground systems of the space systems segment. The primary objective was to determine the feasibility of migrating to PQC with existing and affordable equipment and to validate their effectiveness by comparing their outcome metrics. The findings indicate that PQC migration may have its own pitfalls like higher processing delays, more power usage etc, but can still facilitate meaningful and substantial contributions to systems where security remains the top priority, despite the lower metrics compared to classical cryptography.

Through this work, we have proposed the required study, evaluation methodology and implementation details along with the code snapshots for evaluating the PQC transition metrics in the already existing ground systems. Further we have also given the compiled list of non-technical challenges to be taken care of while involved in the PQC migration. PQC is currently an emerging research field in cryptography and security due to its higher security than the presently operational classical algorithms and its resistance to attacks even by powerful quantum computers in future.

5.2 Future Work

After several years of research, now the PQC landscape reflects steady maturation, with standardization by NIST, performance validation, and initial deployments becoming tangible. Yet, the road ahead includes significant hurdles. Resource constraints, integration complexity, and implementation security gaps pose real barriers to broad adoption. Hybrid transitional strategies and cryptographic-agility (i.e., ability to switch algorithms as needed) are emerging as pragmatic pathways. Embedding PQC into critical systems will demand cross-disciplinary coordination among cryptographers, developers, vendors, and policymakers.

Some future challenges include, developing and benchmarking lightweight PQC variants suitable for IoT/embedded systems, creating side-channel-resistant libraries and using formal methods to validate the custom PQC code, building systems with cryptographic-agility that can seamlessly update or swap cryptographic primitives as standards evolve, expanding the practical hybrid protocols for broader adoption into already existing communication frameworks etc. Post-Quantum Cryptography has established itself as a vital pillar in the future of cyber security. Real-world adoption is in nascent stage now but will be promising towards the future tech landscape.

Organisations have to build a team of talented, well-trained, and dedicated engineers and managers who are focused on delivering reliable and efficient PQC solutions and also leverage advanced automation tools to support key functions such as discovery, replacement, CBOM (Cryptographic Bill-of-Materials) and SBOM (Software Bill-of-Materials) maintenance, verification, and continuous testing towards migrating all existing modules to PQC secure modules. This integrated approach helps ensure operational efficiency, consistency, and high-quality outcomes across the development and maintenance lifecycle. Performance studies in realistic, resource-constrained settings reveal that not all PQC algorithms are feasible without optimization trade-off (i.e) algorithm selection must balance security with efficiency. Implementation security, particularly against side-channel attacks, remains a critical challenge even for theoretically sound schemes.

As PQC continues to evolve, it is important to develop and integrate advanced techniques that can preemptively address potential side-channel vulnerabilities like Timing attacks, Power Analysis Attacks etc. In sum, while PQC offers a path to future-proof digital security, achieving it securely and efficiently will require advances in optimized algorithms, hardened implementations, deployment infrastructure, organisational capabilities and cryptographic agility frameworks.

APPENDIX A

CODE ATTACHMENTS

A.1 Code for MLKEM

```
1  public class MLKEM {
2
3      private static final String KEM_ALGORITHM = "Kyber";
4      private static final AlgorithmParameterSpec KEM_PARAMETER_SPEC =
           KyberParameterSpec.kyber1024;
5      private static final String PROVIDER = "BCPQC";
6      private static final String ENCRYPTION_ALGORITHM = "AES";
7      private static final Integer ENCRYPTION_ALGORITHM_STRENGTH = 256;
8      private static final String MODE_PADDING = "AES/ECB/PKCS5Padding"
           ;
9
10     public static byte[] encrypt(byte[] plainTextBytes, byte[] key)
           throws Exception {
11         SecretKeySpec secretKey = new SecretKeySpec(key,
               ENCRYPTION_ALGORITHM);
12         Cipher cipher = Cipher.getInstance(MODE_PADDING);
13         cipher.init(Cipher.ENCRYPT_MODE, secretKey);
14         byte[] encryptedBytes = cipher.doFinal(plainTextBytes);
15         return encryptedBytes;
16     }
17
18     public static byte[] decrypt(byte[] encryptedBytes, byte[] key)
           throws Exception {
19         System.out.println(key.length);
20         SecretKeySpec secretKey = new SecretKeySpec(key,
               ENCRYPTION_ALGORITHM);
21         Cipher cipher = Cipher.getInstance(MODE_PADDING);
22         cipher.init(Cipher.DECRYPT_MODE, secretKey);
23         byte[] decryptedBytes = cipher.doFinal(encryptedBytes);
24         return decryptedBytes;
25     }
26
27     private static KeyPair generateKeyPair()
28             throws NoSuchAlgorithmException, NoSuchProviderException,
                   InvalidAlgorithmParameterException {
29         KeyPairGenerator keyPairGenerator = KeyPairGenerator.
               getInstance(KEM_ALGORITHM, PROVIDER);
30         keyPairGenerator.initialize(KEM_PARAMETER_SPEC, new
               SecureRandom());
31         return keyPairGenerator.generateKeyPair();
32     }
33
34     private static SecretKeyWithEncapsulation generateSecretKeySender
           (PublicKey publicKey)
35             throws NoSuchAlgorithmException, NoSuchProviderException,
```

```
                InvalidAlgorithmParameterException {
36
37          KeyGenerator keyGenerator = KeyGenerator.getInstance(
                KEM_ALGORITHM, PROVIDER);
38          KEMGenerateSpec kemGenerateSpec =
39                  new KEMGenerateSpec.Builder(publicKey,
                        ENCRYPTION_ALGORITHM,
                        ENCRYPTION_ALGORITHM_STRENGTH).build();
40          keyGenerator.init(kemGenerateSpec);
41          return (SecretKeyWithEncapsulation) keyGenerator.generateKey
                ();
42      }
43
44      private static SecretKeyWithEncapsulation
            generateSecretKeyReciever(PrivateKey privateKey, byte[]
            encapsulation)
45              throws NoSuchAlgorithmException, NoSuchProviderException,
                    InvalidAlgorithmParameterException {
46
47          KeyGenerator keyGenerator = KeyGenerator.getInstance(
                KEM_ALGORITHM, PROVIDER);
48          KEMExtractSpec kemExtractSpec =
49                  new KEMExtractSpec.Builder(privateKey, encapsulation,
                        ENCRYPTION_ALGORITHM,
                        ENCRYPTION_ALGORITHM_STRENGTH).build();
50          keyGenerator.init(kemExtractSpec);
51
52          return (SecretKeyWithEncapsulation)keyGenerator.generateKey()
                ;
53      }
54
55      public static void main(String[] args) throws Exception {
56          Security.addProvider(new BouncyCastleProvider());
57          Security.addProvider(new BouncyCastlePQCProvider());
58
59          byte[] originalBytes = Files.readAllBytes(Paths.get("F:\\
                TestFiles\\file50M.zip"));
60
61          Long start, end;
62          start = System.currentTimeMillis();
63
64          // Alice
65          KeyPair keyPair = generateKeyPair();
66          System.out.println("KEM Algorithm: " + keyPair.getPublic().
                getAlgorithm());
67          System.out.println("Public Key length: " + keyPair.getPublic
                ().getEncoded().length);
68          System.out.println("Private Key length: " + keyPair.
                getPrivate().getEncoded().length);
69          // Alice send the Public Key to Bob
70
71          // Bob
72          SecretKeyWithEncapsulation initKeyWithEnc =
                generateSecretKeySender(keyPair.getPublic());
73          byte[] encapsulation = initKeyWithEnc.getEncapsulation();
74          System.out.println("Shared Secret created by Sender: " + Hex.
```

```
            toHexString(initKeyWithEnc.getEncoded()));
75          System.out.println("Length of encapsulated shared secret: " +
                encapsulation.length);
76          byte[] encryptedBytes = encrypt(originalBytes, initKeyWithEnc
                .getEncoded());
77          // Bob send the Encapsulation to Alice
78
79          // Alice
80          SecretKeyWithEncapsulation recKeyWithEnc =
                generateSecretKeyReciever(keyPair.getPrivate(),
                encapsulation);
81          System.out.println("Shared Secret decapsulated by Receiver: "
                + Hex.toHexString(recKeyWithEnc.getEncoded()));
82          byte[] decryptedBytes = decrypt(encryptedBytes, recKeyWithEnc
                .getEncoded());
83
84          end = System.currentTimeMillis();
85          System.out.println("Time taken: " + ((end - start) / 1000.0)
                + " seconds");
86
87          System.out.println("Original Length: " + originalBytes.length
                );
88          System.out.println("Encrypted Length: " + encryptedBytes.
                length);
89          System.out.println("Decrypted Length: " + decryptedBytes.
                length);
90          System.out.println("Original == Decrypted ? => " + Arrays.
                equals(originalBytes, decryptedBytes));
91      }
92  }
```

A.2 Code for MLDSA

```
1  public class MLDSA {
2
3      public static void main(String[] args) throws Exception {
4          Security.addProvider(new BouncyCastleProvider());
5          Security.addProvider(new BouncyCastlePQCProvider());
6
7          byte[] originalBytes = Files.readAllBytes(Paths.get("F:\\
               TestFiles\\file50M.zip"));
8
9          Long start, end;
10         Long overall_start, overall_end;
11         overall_start = System.currentTimeMillis();
12
13         System.out.println("Key Generation started:");
14         start = System.currentTimeMillis();
15         KeyPairGenerator kpg = KeyPairGenerator.getInstance("
               Dilithium", "BCPQC");
16         kpg.initialize(DilithiumParameterSpec.dilithium5);
17         KeyPair kp = kpg.generateKeyPair();
18         System.out.println("Key Generation ended:");
19         end = System.currentTimeMillis();
20         System.out.println("Time taken for Key Generation:" + ((end
               - start) / 1000.0) + "seconds");
21
22         System.out.println("Signature Generation started:");
23         start = System.currentTimeMillis();
24         Signature sig = Signature.getInstance("Dilithium", "BCPQC");
25         sig.initSign(kp.getPrivate());
26         sig.update(originalBytes);
27         byte[] signature = sig.sign();
28         System.out.println("Signature Generation ended:");
29         end = System.currentTimeMillis();
30         System.out.println("Signature Length:" + signature.length);
31         System.out.println("Time taken for Signature Generation:" +
               ((end - start) / 1000.0) + "seconds");
32
33         System.out.println("Signature Validation started:");
34         start = System.currentTimeMillis();
35         sig.initVerify(kp.getPublic());
36         sig.update(originalBytes);
37         boolean isVerified = sig.verify(signature);
38         System.out.println("Signature Verified:" + isVerified);
39         System.out.println("Signature Validation ended:");
40         end = System.currentTimeMillis();
41         System.out.println("Time taken for Signature Validation:" +
               ((end - start) / 1000.0) + "seconds");
42
43         overall_end = System.currentTimeMillis();
44         System.out.println("Overall Time taken:" + ((overall_end -
               overall_start) / 1000.0) + "seconds");
45     }
46 }
```

A.3 Code for SLHDSA

```
1  public class SLHDSA {
2
3      public static byte[] GenerateSignature(byte[] originalBytes,
           KeyPair keys) throws SignatureException,
           UnsupportedEncodingException, InvalidKeyException,
           NoSuchAlgorithmException, NoSuchProviderException {
4          Signature sign = Signature.getInstance("SPHINCSPLUS", "BCPQC"
               );
5          sign.initSign(keys.getPrivate());
6          sign.update(originalBytes);
7          byte[] signature = sign.sign();
8          return signature;
9      }
10
11     public static boolean ValidateSignature(byte[] originalBytes,
           KeyPair pair, byte[] signature) throws SignatureException,
           InvalidKeyException, UnsupportedEncodingException,
           NoSuchAlgorithmException, NoSuchProviderException {
12         Signature verify = Signature.getInstance("SPHINCSPLUS", "
               BCPQC");
13         verify.initVerify(pair.getPublic());
14         verify.update(originalBytes);
15         return verify.verify(signature);
16     }
17
18     public static KeyPair GenerateKeys(SPHINCSPlusParameterSpec
           sPHINCSPlusParameterSpec) throws NoSuchAlgorithmException,
           NoSuchProviderException, InvalidAlgorithmParameterException {
19         KeyPairGenerator g = KeyPairGenerator.getInstance("
               SPHINCSPLUS", "BCPQC");
20         g.initialize(sPHINCSPlusParameterSpec);
21         return g.generateKeyPair();
22     }
23
24     public static void main(String[] args) throws Exception {
25         Security.addProvider(new BouncyCastleProvider());
26         Security.addProvider(new BouncyCastlePQCProvider());
27
28         byte[] originalBytes = Files.readAllBytes(Paths.get("F:\\
               TestFiles\\file50M.zip"));
29
30         Long start, end;
31         Long overall_start, overall_end;
32         overall_start = System.currentTimeMillis();
33
34         System.out.println("Key■Generation■started:■");
35         start = System.currentTimeMillis();
36         KeyPair keys = GenerateKeys(SPHINCSPlusParameterSpec.
               sha2_256s);
37         System.out.println("Key■Generation■ended:■");
38         end = System.currentTimeMillis();
39         System.out.println("Time■taken■for■Key■Generation:■" + ((end
               - start) / 1000.0) + "■seconds");
```

```
40
41          System.out.println("Signature Generation started: ");
42          start = System.currentTimeMillis();
43          byte[] signature = GenerateSignature(originalBytes, keys);
44          System.out.println("Signature Generation ended: ");
45          end = System.currentTimeMillis();
46          System.out.println("Signature Length: " + signature.length);
47          System.out.println("Time taken for Signature Generation: " +
                ((end - start) / 1000.0) + " seconds");
48
49          System.out.println("Signature Validation started: ");
50          start = System.currentTimeMillis();
51          boolean isValidated = ValidateSignature(originalBytes, keys,
                signature);
52          System.out.println("Signature Verified: " + isValidated);
53          System.out.println("Signature Validation ended: ");
54          end = System.currentTimeMillis();
55          System.out.println("Time taken for Signature Validation: " +
                ((end - start) / 1000.0) + " seconds");
56
57          overall_end = System.currentTimeMillis();
58          System.out.println("Overall Time taken: " + ((overall_end -
                overall_start) / 1000.0) + " seconds");
59      }
60  }
```

A.4 Code for Maven Project POM XML

```
1  <?xml version="1.0" encoding="UTF-8"?>
2  <project xmlns="http://maven.apache.org/POM/4.0.0"
3           xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4           xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http:
                //maven.apache.org/xsd/maven-4.0.0.xsd">
5
6
7  <!--    mvn clean && mvn install -Dmaven.wagon.http.ssl.insecure=true
       -Dmaven.wagon.http.ssl.allowall=true -->
8  <!--    mvn dependency:tree -->
9  <!--    mvn clean verify -->
10 <!--    mvn clean install -U -->
11
12
13     <modelVersion>4.0.0</modelVersion>
14     <groupId>com.project</groupId>
15     <artifactId>MTech-3</artifactId>
16     <version>1.0-SNAPSHOT</version>
17     <packaging>jar</packaging>
18     <name>${project.artifactId}</name>
19 <!--    <url>http://maven.apache.org</url> -->
20
21
22     <repositories>
23         <repository>
24             <id>local-repo</id>
25             <url>file://${project.basedir}/repo</url>
26         </repository>
27     </repositories>
28
29     <properties>
30         <maven.compiler.source.version>11</maven.compiler.source.
               version>
31         <maven.compiler.target.version>11</maven.compiler.target.
               version>
32         <eclipse.collections.version>11.1.0</eclipse.collections.
               version>
33         <jmh-core.version>1.37</jmh-core.version>
34         <jmh-generator.version>1.37</jmh-generator.version>
35         <uberjar.name>java-collection-impls-benchmark</uberjar.name>
36         <shade.plugin.version>3.2.4</shade.plugin.version>
37     </properties>
38
39
40     <build>
41
42         <plugins>
43
44             <plugin>
45                 <groupId>org.apache.maven.plugins</groupId>
46                 <artifactId>maven-compiler-plugin</artifactId>
47                 <configuration>
48                     <source>${maven.compiler.source.version}</source>
```

```
49                      <target>${maven.compiler.target.version}</target>
50                  </configuration>
51  <!--                <configuration>-->
52  <!--                    <release>21</release> &lt;!&ndash; Preferred
        for Java 9+ &ndash;&gt;-->
53  <!--                </configuration>-->
54              </plugin>
55
56              <plugin>
57                  <groupId>org.apache.maven.plugins</groupId>
58                  <artifactId>maven-shade-plugin</artifactId>
59                  <version>${shade.plugin.version}</version>
60                  <executions>
61                      <execution>
62                          <phase>package</phase>
63                          <goals>
64                              <goal>shade</goal>
65                          </goals>
66                          <configuration>
67                              <finalName>${uberjar.name}</finalName>
68                              <transformers>
69                                  <transformer implementation="org.
                                        apache.maven.plugins.shade.
                                        resource.
                                        ManifestResourceTransformer">
70                                      <mainClass>org.openjdk.jmh.Main</
                                            mainClass>
71                                  </transformer>
72                                  <transformer implementation="org.
                                        apache.maven.plugins.shade.
                                        resource.
                                        ServicesResourceTransformer"/>
73                              </transformers>
74                              <filters>
75                                  <filter>
76                                      <artifact>*:*</artifact>
77                                      <excludes>
78                                          <exclude>META-INF/*.SF</
                                                exclude>
79                                          <exclude>META-INF/*.DSA</
                                                exclude>
80                                          <exclude>META-INF/*.RSA</
                                                exclude>
81                                      </excludes>
82                                  </filter>
83                              </filters>
84                          </configuration>
85                      </execution>
86                  </executions>
87              </plugin>
88
89          </plugins>
90
91      </build>
92
93      <dependencies>
```

```
94          <dependency>
95              <groupId>org.bouncycastle</groupId>
96              <artifactId>bcprov-jdk18on</artifactId>
97               <version>1.84</version>
98          </dependency>
99
100 <!--        <dependency>-->
101 <!--            <groupId>org.bouncycastle</groupId>-->
102 <!--            <artifactId>bcprov-jdk15to18</artifactId>-->
103 <!--            <version>1.78</version>-->
104 <!--        </dependency>-->
105
106         <dependency>
107             <groupId>javax.xml.bind</groupId>
108             <artifactId>jaxb-api</artifactId>
109             <version>2.3.1</version>
110         </dependency>
111
112         <dependency>
113             <groupId>commons-io</groupId>
114             <artifactId>commons-io</artifactId>
115             <version>2.22.0</version>
116         </dependency>
117
118         <dependency>
119             <groupId>io.github.msg-systems</groupId>
120             <artifactId>jpowermonitor</artifactId>
121             <version>2.0.0</version>
122         </dependency>
123
124         <dependency>
125             <groupId>org.openjdk.jmh</groupId>
126             <artifactId>jmh-core</artifactId>
127             <version>${jmh-core.version}</version>
128         </dependency>
129
130         <dependency>
131             <groupId>org.openjdk.jmh</groupId>
132             <artifactId>jmh-generator-annprocess</artifactId>
133             <version>${jmh-generator.version}</version>
134             <scope>provided</scope>
135         </dependency>
136
137         <dependency>
138             <groupId>concurrent</groupId>
139             <artifactId>concurrent</artifactId>
140             <version>1.3.4</version>
141             <scope>compile</scope>
142         </dependency>
143
144         <dependency>
145             <groupId>org.eclipse.collections</groupId>
146             <artifactId>eclipse-collections</artifactId>
147             <version>${eclipse.collections.version}</version>
148         </dependency>
149
```

```
150         <dependency>
151             <groupId>org.eclipse.collections</groupId>
152             <artifactId>eclipse-collections-api</artifactId>
153             <version>${eclipse.collections.version}</version>
154         </dependency>
155 
156         <dependency>
157             <groupId>joularjx</groupId>
158             <artifactId>joularjx</artifactId>
159             <version>3.1.0</version>
160             <scope>system</scope>
161             <systemPath>${project.basedir}/jars/joularjx-3.1.0.jar</
                    systemPath>
162         </dependency>
163 
164         <dependency>
165             <groupId>jRAPL</groupId>
166             <artifactId>jRAPL</artifactId>
167             <version>1.0</version>
168             <scope>system</scope>
169             <systemPath>${project.basedir}/jars/jRAPL-1.0.jar</
                    systemPath>
170         </dependency>
171 
172     </dependencies>
173 
174 </project>
```

A.5 Code for Visualisation

```
1  ######################################################
2  # MLKEM
3  ######################################################
4
5  import matplotlib.pyplot as plt
6  import numpy as np
7
8  title = "MLKEM"
9  x = [50, 100, 200, 300, 400, 500, 600, 700, 800, 900]
10
11 title1 = "MLKEM-0512-AES-128"
12 title2 = "MLKEM-0768-AES-192"
13 title3 = "MLKEM-1024-AES-256"
14
15 #1
16 filename1 = title1 + ".txt"
17 f1 = open(filename1, "r")
18 y1 = []
19 for line in f1:
20         item=line.split()
21         y1.append(float(item[0]))
22 plt.plot(x, y1, marker='o', linestyle='-', color='r', label=title1)
23
24 #2
25 filename2 = title2 + ".txt"
26 f2 = open(filename2, "r")
27 y2 = []
28 for line in f2:
29         item=line.split()
30         y2.append(float(item[0]))
31 plt.plot(x, y2, marker='x', linestyle='-', color='g', label=title2)
32
33 #3
34 filename3 = title3 + ".txt"
35 f3 = open(filename3, "r")
36 y3 = []
37 for line in f3:
38         item=line.split()
39         y3.append(float(item[0]))
40 plt.plot(x, y3, marker='s', linestyle='-', color='b', label=title3)
41
42
43 plt.title(title)
44 plt.xlabel('File Size (in MB)')
45 plt.ylabel('Time Taken (in Seconds)')
46 plt.legend()
47
48 plt.savefig("images/" + title + ".png")
49 plt.show()
50 f1.close()
51 f2.close()
52 f3.close()
53 ######################################################
```

```
1  ######################################################
2  # MLDSA
3  ######################################################
4
5  import matplotlib.pyplot as plt
6  import numpy as np
7
8  title = "MLDSA"
9  x = [50, 100, 200, 300, 400, 500, 600, 700, 800, 900]
10
11 title1 = "MLDSA-44"
12 title2 = "MLDSA-65"
13 title3 = "MLDSA-87"
14
15 #1
16 filename1 = title1 + ".txt"
17 f1 = open(filename1, "r")
18 y1 = []
19 for line in f1:
20         item=line.split(",")
21         y1.append(float(item[3]))
22 plt.plot(x, y1, marker='o', linestyle='-', color='r', label=title1)
23
24 #2
25 filename2 = title2 + ".txt"
26 f2 = open(filename2, "r")
27 y2 = []
28 for line in f2:
29         item=line.split(",")
30         y2.append(float(item[3]))
31 plt.plot(x, y2, marker='x', linestyle='-', color='g', label=title2)
32
33 #3
34 filename3 = title3 + ".txt"
35 f3 = open(filename3, "r")
36 y3 = []
37 for line in f3:
38         item=line.split(",")
39         y3.append(float(item[3]))
40 plt.plot(x, y3, marker='s', linestyle='-', color='b', label=title3)
41
42
43 plt.title(title)
44 plt.xlabel('File Size (in MB)')
45 plt.ylabel('Time Taken (in Seconds)')
46 plt.legend()
47
48 plt.savefig("images/" + title + ".png")
49 plt.show()
50 f1.close()
51 f2.close()
52 f3.close()
53 ######################################################
```

```
1  ######################################################
2  # SLHDSA
3  ######################################################
4
5  import matplotlib.pyplot as plt
6  import numpy as np
7
8  title = "SLHDSA_sha2"
9  # title = "SLHDSA_shake
10 x = [50, 100, 200, 300, 400, 500, 600, 700, 800, 900]
11
12 title1 = "SLHDSA_sha2_128f"
13 title2 = "SLHDSA_sha2_128s"
14 title3 = "SLHDSA_sha2_192f"
15 title4 = "SLHDSA_sha2_192s"
16 title5 = "SLHDSA_sha2_256f"
17 title6 = "SLHDSA_sha2_256s"
18
19 # title1 = "SLHDSA_shake_128f"
20 # title2 = "SLHDSA_shake_128s"
21 # title3 = "SLHDSA_shake_192f"
22 # title4 = "SLHDSA_shake_192s"
23 # title5 = "SLHDSA_shake_256f"
24 # title6 = "SLHDSA_shake_256s"
25
26 #1
27 filename1 = title1 + ".txt"
28 f1 = open(filename1, "r")
29 y1 = []
30 for line in f1:
31         item=line.split(",")
32         y1.append(float(item[3]))
33 plt.plot(x, y1, marker='o', linestyle='-', color='r', label=title1)
34
35 #2
36 filename2 = title2 + ".txt"
37 f2 = open(filename2, "r")
38 y2 = []
39 for line in f2:
40         item=line.split(",")
41         y2.append(float(item[3]))
42 plt.plot(x, y2, marker='s', linestyle='-', color='r', label=title2)
43
44 #3
45 filename3 = title3 + ".txt"
46 f3 = open(filename3, "r")
47 y3 = []
48 for line in f3:
49         item=line.split(",")
50         y3.append(float(item[3]))
51 plt.plot(x, y3, marker='o', linestyle='-', color='g', label=title3)
52
53 #4
54 filename4 = title4 + ".txt"
55 f4 = open(filename4, "r")
56 y4 = []
```

```
for line in f4:
        item=line.split(",")
        y4.append(float(item[3]))
plt.plot(x, y4, marker='s', linestyle='-', color='g', label=title4)

#5
filename5 = title5 + ".txt"
f5 = open(filename5, "r")
y5 = []
for line in f5:
        item=line.split(",")
        y5.append(float(item[3]))
plt.plot(x, y5, marker='o', linestyle='-', color='b', label=title5)

#6
filename6 = title6 + ".txt"
f6 = open(filename6, "r")
y6 = []
for line in f6:
        item=line.split(",")
        y6.append(float(item[3]))
plt.plot(x, y6, marker='s', linestyle='-', color='b', label=title6)


plt.title(title)
plt.xlabel('File Size (in MB)')
plt.ylabel('Time Taken (in Seconds)')
plt.legend()

plt.savefig("images/" + title + ".png")
plt.show()
f1.close()
f2.close()
f3.close()
f4.close()
f5.close()
f6.close()
#####################################################
```

REFERENCES

[1] P. W. Shor. "Algorithms for quantum computation: discrete logarithms and factoring". In: *Proceedings of the 35th Annual Symposium on Foundations of Computer Science*. 1994, pp. 124–134. DOI: 10.1109/SFCS.1994.365700. URL: https://dl.acm.org/doi/10.1109/SFCS.1994.365700.

[2] Peter W. Shor. "Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer". In: *SIAM Review* 41.2 (1999), pp. 303–332. DOI: 10.1137/S0036144598347011. eprint: https://doi.org/10.1137/S0036144598347011. URL: https://doi.org/10.1137/S0036144598347011.

[3] Lov K. Grover. "A fast quantum mechanical algorithm for database search". In: *Proceedings of the Twenty-Eighth Annual ACM Symposium on Theory of Computing*. STOC '96. Philadelphia, Pennsylvania, USA: Association for Computing Machinery, 1996, 212–219. ISBN: 0897917855. DOI: 10.1145/237814.237866. URL: https://doi.org/10.1145/237814.237866.

[4] Charles Bennett and Gilles Brassard. "Quantum cryptography: Public key distribution and coin tossing". In: *Theoretical Computer Science - TCS* 560 (Jan. 1984), pp. 175–179. DOI: 10.1016/j.tcs.2011.08.039.

[5] Artur K. Ekert. "Quantum cryptography based on Bell's theorem". In: *Physical Review Letters* 67 (6 1991), pp. 661–663. DOI: 10.1103/PhysRevLett.67.661. URL: https://link.aps.org/doi/10.1103/PhysRevLett.67.661.

[6] Michael A. Nielsen and Isaac L. Chuang. *Quantum Computation and Quantum Information: 10th Anniversary Edition*. Cambridge University Press, 2010.

[7] *NIST Computer Security Resource Center, "Post Quantum Cryptography PQC Standardization Process"*. URL: https://csrc.nist.gov/pqc-standardization.

[8] *NIST Computer Security Resource Center, "FIPS 203 Module Lattice Based Key Encapsulation Mechanism Standard"*. URL: https://csrc.nist.gov/pubs/fips/203/final.

[9] *NIST Computer Security Resource Center, "FIPS 204 Module Lattice Based Digital Signature Standard"*. URL: https://csrc.nist.gov/pubs/fips/204/final.

[10] *NIST Computer Security Resource Center, "FIPS 205 Stateless Hash Based Digital Signature Standard"*. URL: https://csrc.nist.gov/pubs/fips/205/final.

[11] Oded Regev. "On lattices, learning with errors, random linear codes, and cryptography". In: *Proceedings of the Thirty-Seventh Annual ACM Symposium on Theory of Computing*. STOC '05. New York, NY, USA: Association for Computing Machinery, 2005, pp. 84–93. DOI: 10.1145/1060590.1060603.

[12] Kanza Dekkaki, Igor Tasic, and Maria-Dolores Cano. "Exploring Post-Quantum Cryptography: Review and Directions for the Transition Process". In: *Technologies* 12 (Nov. 2024), p. 241. DOI: 10.3390/technologies12120241.

[13] Bharat Rawal and Anjan Biswas. "A Comprehensive Survey of Post-Quantum Cryptography and Its Implications". In: 5 (2 July 2024), pp. 256–269. DOI: 10.37256/est.5220244169.

[14] Shengbo Xu, Ana Vazquez Alonso, Phil Eisen, and Jacques Légaré. "A Preliminary Study of PQC Implementations for Satellite Communication Networks". In: *Proceedings of the ESA Conference* (May 2024). URL: https://atpi.eventsair.com/24a06---3s2024/.

[15] Sergey Yunakovsky, M. Kot, Nikolay Pozhar, Denis Nabokov, Mikhail Kudinov, Anton Guglya, Evgeniy Kiktenko, Ekaterina Kolycheva, Alexander Borisov, and A. Fedorov. "Towards security recommendations for public-key infrastructures for production environments in the post-quantum era". In: *EPJ Quantum Technology* 8 (Dec. 2021). DOI: 10.1140/epjqt/s40507-021-00104-z.

[16] Friday O. Ikwuogu, Joy Agbesi, Fejiro Eni, Damilola Hannah, and Justin Njimgou Zeyeum. "Quantum-Resilient Infrastructure: Migrating US Financial Payment System to Post-Quantum Cryptography (PQC) Standards to Prevent 'Harvest Now, Decrypt Later' Attacks". In: *International Journal of Computer Applications Technology and Research* 13 (Dec. 2024), pp. 198–213. DOI: 10.7753/IJCATR1312.1016.